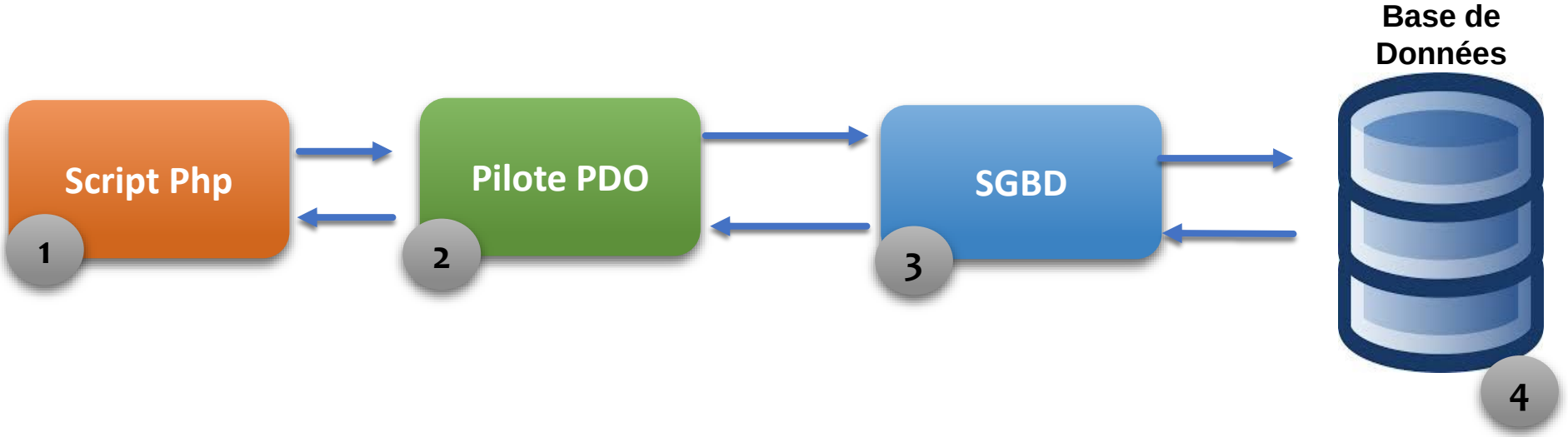
Développement Web:
Javascript & PHP

PHP: PARTIE 4

ACCÈS AUX BASES DE DONNÉES

CONNEXION AUX BASES DE DONNES

- Le script PHP (1) ne dialogue pas directement avec le SGBD (Système de Gestion de Bases de Données) (3).
- On utilise un intermédiaire appelé pilote de SGBD ou encore driver de SGBD (un driver pour chaque SGBD).
- PHP fournit une interface standard pour ces pilotes, l'interface PDO (PHP Data Objects).



Se connecter à une base de données MySql en Php

- 3 moyens existent pour connecter une application Php et une base MySql
  - L'extension MySQL (dépréciée)
  - L'extension MySQLi (MySQL improved) ;
  - L'extension PDO (PHP Data Objects).

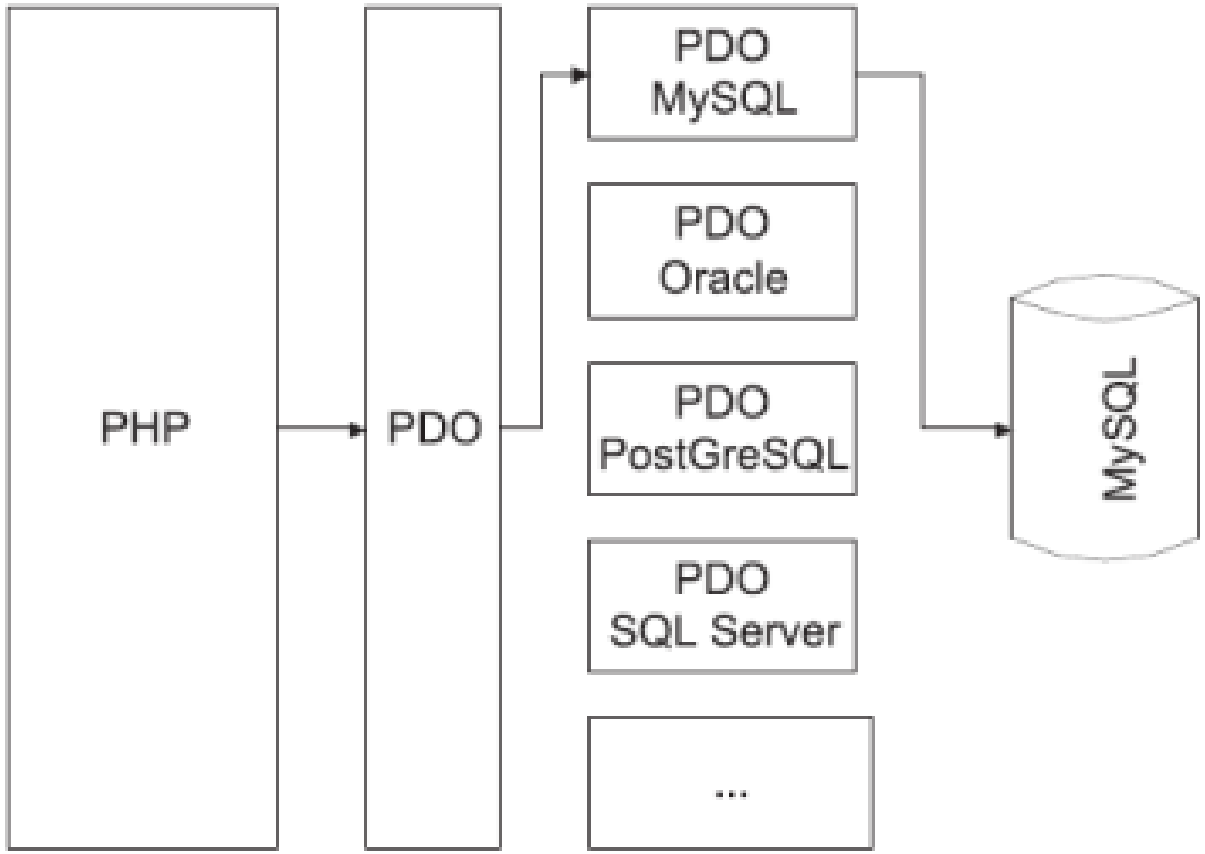
- L'extension MySQLi ne fonctionne qu'avec les bases de données MySQL tandis que PDO fonctionne avec différents systèmes de bases de données (dont les plus utilisés tels que Oracle , SQL Server, etc.).

PDO

- PDO (**P**HP **D**ata **O**bjects): fournit une interface d'abstraction pour l'accès aux données d'une base et non pas une abstraction de la base de données. (Il n'y a pas une réécriture de SQL).
- Elle vous permet d'utiliser les mêmes fonctions pour exécuter des requêtes ou récupérer les données quel que soit la base de données utilisée
- PDO supporte la plupart de bases de données telles que MySQL, Oracle, PostgreSQL, etc.
- PDO est fourni avec PHP 5.1 et requiert les nouvelles fonctionnalités OO fournies par PHP 5 et donc, ne fonctionne pas avec les versions antérieures de PHP.

PDO

- Architecture des drivers PDO



BD supportées

Nom du driver	Bases de données supportées
PDO_DBLIB	FreeTDS / Microsoft SQL Server / Sybase
PDO_FIREBIRD	Firebird/Interbase 6
PDO_IBM	IBM DB2
PDO_INFORMIX	IBM Informix Dynamic Server
PDO_MYSQL	MySQL 3.x/4.x/5.x
PDO_OCI	Oracle Call Interface
PDO_ODBC	ODBC v3 (IBM DB2, unixODBC et win32 ODBC)
PDO_SQLITE	SQLite 3 et SQLite 2
PDO_PGSQL	PostgreSQL
PDO_4D	4D

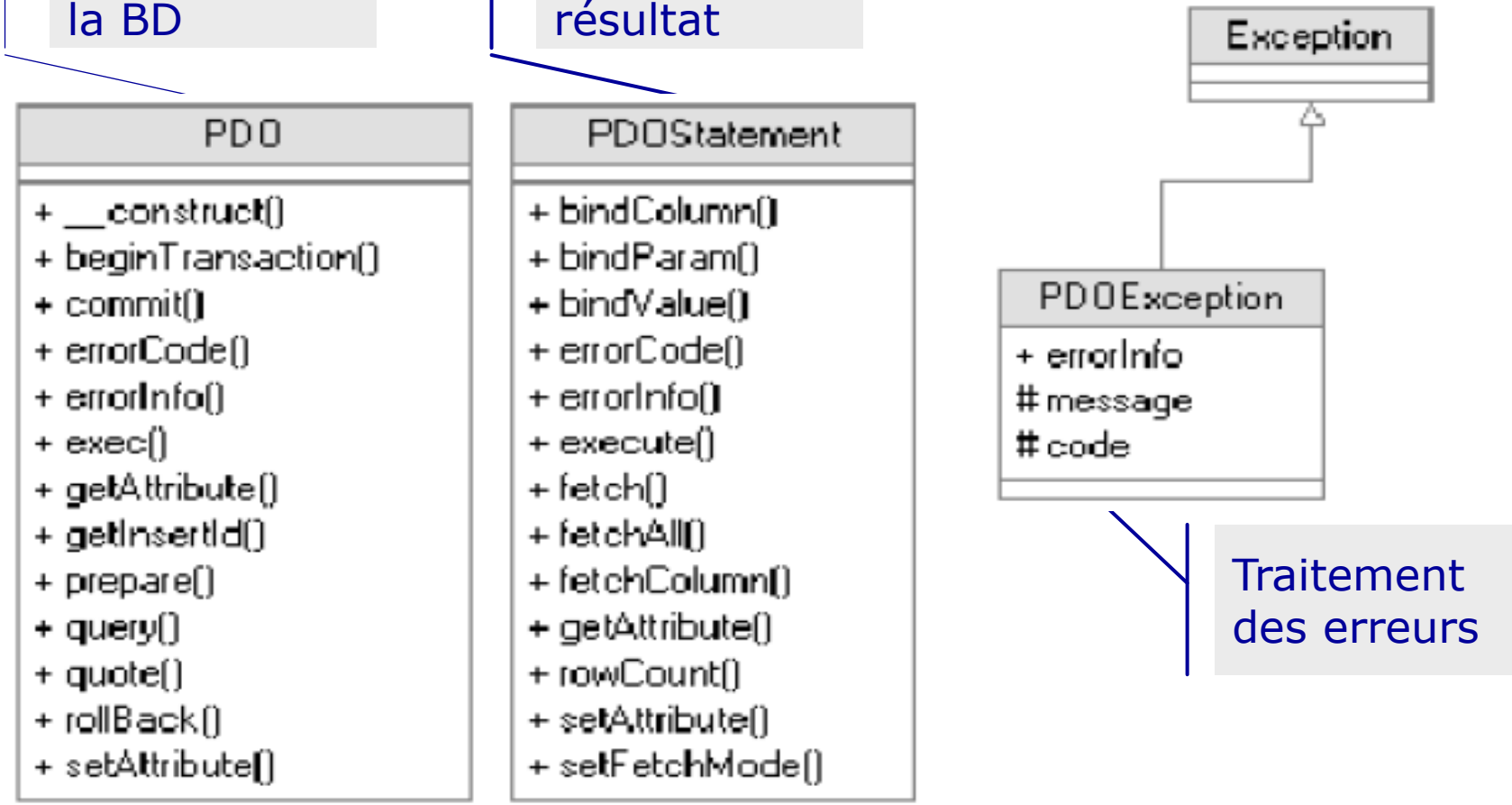
PDO : les classes à utiliser

Classe	Rôle
PDO	gère l'accès aux SGBD ainsi que les fonctionnalités de base : Connexions, lancement des requêtes…
PDOStatement	La classe gère une liste de résultats d'une requête Sql
PDOException	Une classe d'exception personnalisée interne pour PDO

Structure des classes de PDO

Lien entre script PHP et la BD

Les requêtes et leur résultat



Traitement des erreurs

Principe des exceptions

- Les exceptions permettent de gérer les erreurs d'exécution.
- Une exception définit un espace dans lequel les erreurs sont détectées.
- Les exceptions permettent de mieux organiser la gestion des erreurs et des événements.
- **Exemple:**

```php
<?php
    try {
            // traitement
        }
    catch (PDOException $e) {
            // traitement d'une erreur base de données
        }
    catch (Exception $e) {
            // Traitement d'une erreur inconnue    }
?>
```

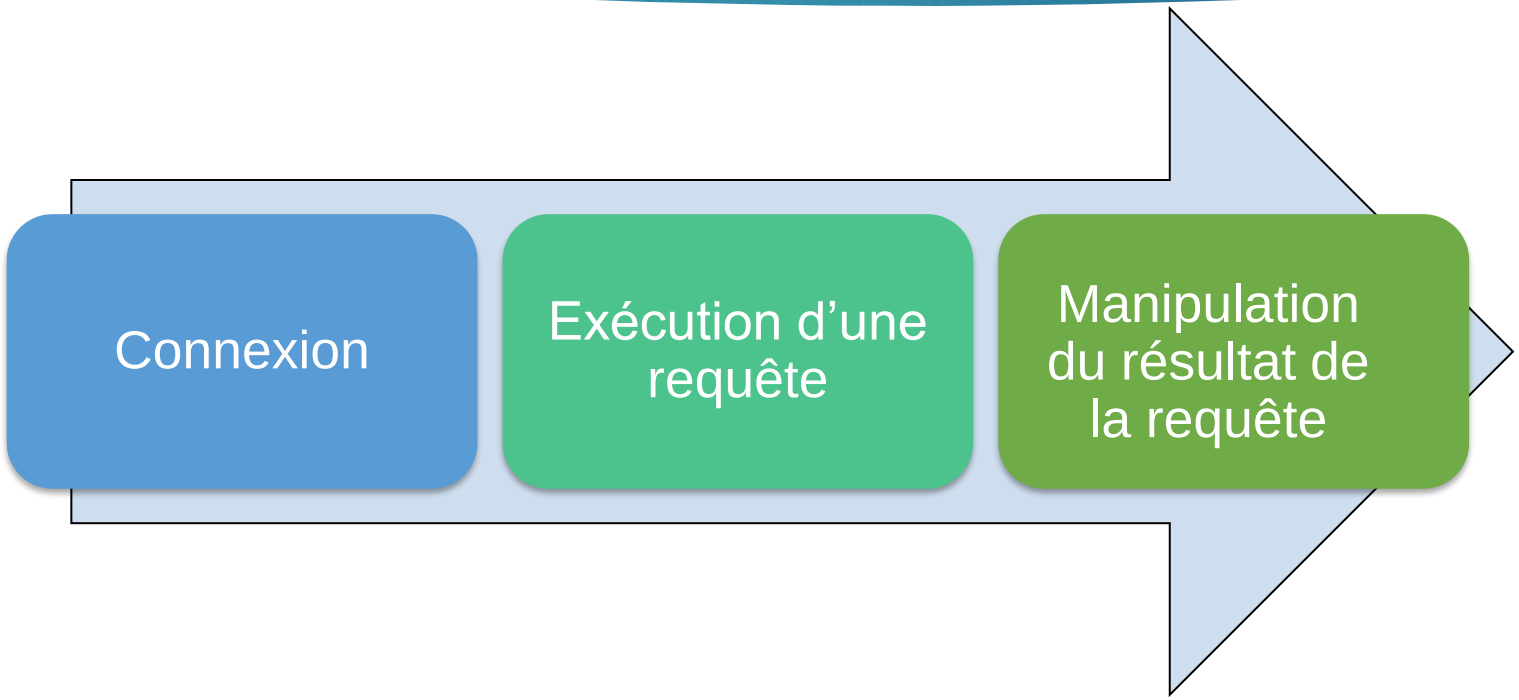
Classes prédéfinies

- **PDO**
  - Connexion PHP/Base de données
  - __construct()
  - exec(), prepare(), query()
  - errorCode(), errorInfo()
  - getAttributes(), setAttribute()
  - lastInsertId(), quote()
  - beginTransaction()
  - commit(), rollBack()
  - getAvailableDrivers()

Classes prédéfinies

- **PDOStatement**
  - requête préparée, jeu de résultats
  - bindColumn(), bindParam(), bindValue(), closeCursor()
  - errorCode(), errorInfo()
  - fetch(), fetchAll(), fetchColumn(), fetchObject(), setFetchMode(), nextRowset()
  - rowCount(), columnCount(), getColumnMeta()
  - getAttribute(), setAttribute()
  - execute()
  - debugDumpParams()

Processus de manipulation des BDs



Connexion

- **Instanciation d'un objet PDO**

```
$cnx=new PDO(DSN [, user [, pass [, options]]]);
```

  - DSN : Data Source Name
    - nom_du_driver:syntaxe_spécifique_au_driver
    - Exemple : `mysql:host=localhost;dbname=ma_base`
  - user : nom d'utilisateur, pass : mot de passe
  - options : tableau associatif spécifique au driver
    - Exemple :

    array(PDO::ATTR_PERSISTENT => true, PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION)) ;

Gestion des erreurs de connexion

- Exemple

```php
<?php

try {
 $cnx = new PDO($dsn, $user, $pass) ;

 //…Traitement
 $cnx = null ;
}

catch (PDOException $e) {
 echo "Erreur: ".$e->getMessage()."<br>" ;
}

?>
```

Connexion: exemple

- Exemple de script de connexion à une base données « gestion » stockée en local (usernamme="root" et password=""):

```php
<?php
        $servername = "localhost";
        $username = "root";
        $password = "root";
        $base = "gestion";
try {
        $connexion = new PDO("mysql:host=$servername;dbname=$base",$username,$password);
}
catch(PDOException $e)
{       echo "Connection failed: " . $e->getMessage();   }
?>
```

Types de requêtes exécutées avec PDO

- Tout objet de type PDO peut manipuler 2 types de requêtes : **query()** et **exec()**.

query

Pour réaliser une requête qui récupère des données de la BD (SELECT)

exec

Pour réaliser les trois autres requêtes comme l'insertion (INSERT) , modification (UPDATE) et, suppression (DELETE) de données

La fonction : query

```
public PDO::query ( string $statement ) : PDOStatement
```

- Exécute une requête SQL,
- Retourne un jeu de résultats en tant qu'objet PDOStatement, FALSE si une erreur survient
- **$statement**
  - La requête SQL à exécuter.
- **Exemple:**

```
$resultat = $cnx->query("SELECT * FRON CLIENT ");
```

La fonction : exec

```
public PDO::exec ( string $statement ) : int
```

- Cette fonction exécute une requête SQL (string)et retourne le nombre de lignes affectées. Si aucune ligne n'est affectée, la fonction retournera 0
- **$statement**
  - La requête SQL à exécuter.
- **Exemple:**

```
$cnx->exec("INSERT INTO CLIENT  VALUES ('ahmed', 20)");
```

BD d'exemple:

- Modèle relationnel

```
Client (id, nom, cin, date_naissance)
```

id	nom	cin	date_naissance
3	bilel	12121212	1980-10-23
4	ridha	23232323	2000-11-16
5	nahed	33333333	1990-11-12
6	nadou	66666666	2005-09-02
7	arial	45454545	1995-07-07

“INSERTION DES DONNÉES DANS UNE TABLE”

SQL d'insertion

- Structure de la requête SQL d'insertion

```
INSERT INTO nom_de_table (nom_colonne1, nom_colonne2, nom_colonne3, …)
VALUES (valeur1, valeur2, valeur3, …)
```

- **Règles de syntaxe à respecter afin que la requête INSERT fonctionne :**
  - Les valeurs de type chaine de caractère (String) doivent être placées entre apostrophes ;
  - La valeur NULL ne doit pas être placée entre apostrophes ;
  - Les valeurs de type numérique ne doivent pas être placées entre apostrophes.
  - Vous devriez avoir autant de valeurs à insérer qu'il y a de colonnes dans votre table.

    Cependant, notez qu'il n'est pas nécessaire de préciser les colonnes possédant un attribut AUTO_INCREMENT ou TIMESTAMP (gérer automatiquement par le SGBD)

SQL d'insertion

- Exemple : ajouter un nouveau client

```php
<?php
   require_once("connexion.php");
   # Créer la chaîne de la requête
$requete ="INSERT INTO Client(nom, cin, date_naissance) VALUES('bilel',11111111,'1980-
10-23')";
   # Exécuter la chaîne de la requête
   $resultat = $connexion->exec($requete);
   if($resultat == 0)
       echo "echec d'ajout";
    else
        echo "ajout avec succès d'un client";
?>
```

“Mise à jour des données dans une table”

SQL update

- Structure de la requête SQL de mise à jour

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

SQL update

- Exemple : Changer le nom du client d'id 1 à ridha

```php
<?php
   require_once("connexion.php");
   # Créer la chaîne de la requête
   $requete ="UPDATE CLIENT set nom = 'ridha' where id=1";
   # Exécuter la chaîne de la requête
   $resultat = $connexion->exec($requete);
   if($resultat == 0)
       echo "echec de maj";
    else
        echo "maj avec succès d'un client";
?>
```

“SUPPRIMER DES DONNÉES DANS UNE TABLE”

SQL delete

- Structure de la requête SQL de suppression

```
DELETE FROM table_name WHERE condition;
```

SQL update

- Exemple : supprimer le client ayant un cin = 11111111

```php
<?php
   require_once("connexion.php");
   # Créer la chaîne de la requête
   $requete =" DELETE FROM Client WHERE cin = 11111111";
   # Exécuter la chaîne de la requête
   $resultat = $connexion->exec($requete);
   if($resultat == 0)
       echo "echec de suppression";
    else
        echo "delete avec succès d'un client";
?>
```

“Exploitation du résultat d’une requête SELECT”

Exécution d'une requête select

```
PDOStatement  PDO::query ( string statement )
```

- Exemple

```
<?php

try {
 $cnx = new PDO ($dsn, $user, $pass)  ;
 $pdostat = $cnx->query("SELECT * FROM clients") ;
}

catch (PDOException $e) {
 echo "Erreur: ".$e->getMessage()."<br>" ;
}
?>
```

Nombre de lignes et de colonnes

- **rowCount()** : permet de retourner le nombre de lignes d'une requête
- **columnCount()** : permet de retourner le nombre de colonnes
- Exemple :

```
$statement = $connexion->query("select * from CLIENT ");
// afficher le nombre de lignes retournées
echo $statement->rowCount();
```

Exploitation des résultats d'une requête

- **2 étapes:**
- Récupération des données ligne à ligne; Une ligne peut être :
  - un tableau indexé
  - un tableau associatif
  - un tableau mixte (par défaut)
  - un objet anonyme/d'une classe définie par l'utilisateur
- Récupération des données d'une colonne

Méthodes d'exploitation des données

- PDO propose deux façons pour exploiter les données après une requête select :

fetch

Permet une **lecture séquentielle** du résultat. **Récupère la ligne suivante d'un jeu de résultats PDO**.

Cette méthode est très utile pour le traitement de gros résultats. Cependant, vous devrez attendre la fin du traitement avant de faire une autre requête et il n'est pas possible de connaitre le nombre de lignes résultat avant d'avoir parcouru l'intégralité du résultat

fetchAll

retourne l'ensemble de tous les données sous forme de tableau et libère la base de données

Cela occasionne une charge importante au niveau du serveur car la totalité des données sont localisées en mémoire

fetchAll()

- Exemple : Afficher tous les clients

```php
<?php
   require_once("connexion.php");

   # Exécuter le requête SELECT
   $statement = $connexion->query('SELECT * FROM CLIENT');
   # Récupérer tout le résultat
   $resultat = $statement->fetchAll();
   # Parcourir le résultat ligne par ligne
   # Chaque ligne : un tableau associatif des informations récupérées
   foreach($resultat as $client)
   {
       echo "id : ". $client['id'];
       echo "  nom : ". $client['nom'];
       echo "  date naissance  : ". $client['date_naissance'];
       echo "<br>";
   }
?>
```

Chaque ligne est indexée par les noms des colonnes de la requête

fetch ()

- Exemple : Afficher tous les clients

```php
<?php
   require_once("connexion.php");

   # Exécuter le requête SELECT
   $statement = $connexion->query('SELECT * FROM CLIENT’);

   # Récupérer le résultat ligne par ligne jusqu’à la fin des données
   while($client = $statement->fetch())
   {
       echo "id : ". $client['id'];
       echo "  nom : ". $client['nom'];
       echo "  date naissance  : ". $client['date_naissance'];
       echo "<br>";
   }
?>
```

Modes de récupération des données

- **PDO::FETCH_ASSOC**
  - retourner chaque ligne dans un tableau indexé par les noms des colonnes ,comme elles sont retournées dans le jeu de résultats correspondant.
- **PDO::FETCH_NUM**
  - retourner chaque ligne dans un tableau indexé par le numéro des colonnes comme elles sont retournées dans le jeu de résultats correspondant, en commençant à 0.
- **PDO::FETCH_BOTH**
  - retourner chaque ligne dans un tableau indexé par les noms des colonnes ainsi que leurs numéros, comme elles sont retournées dans le jeu de résultats correspondant, en commençant à 0.
- **PDO::FETCH_OBJ**
  - retourner chaque ligne dans un objet avec les noms de propriétés correspondant aux noms des colonnes comme elles sont retournées dans le jeu de résultats

Modes de récupération des données

- Exemple : Afficher tous les clients avec le mode PDO::FETCH_NUM

```php
<?php
    require_once("connexion.php");

    # Exécuter le requête SELECT
    $statement = $connexion->query('SELECT * FROM CLIENT');
    # Récupérer le résultat ligne par ligne
    while($client = $statement->fetch(PDO::FETCH_NUM))
    {
        echo "id : ". $client[0];
        echo "  nom : ". $client[1];
        echo "  date naissance  : ". $client[3];
        echo "<br>";
    }
?>
```

Chaque ligne correspondant à un tableau d'indices correspondant aux numéros des colonnes retournées par la requête

Modes de récupération des données

- Exemple : Afficher tous les clients avec le mode PDO::FETCH_OBJ

```php
<?php
   require_once("connexion.php");

   # Exécuter le requête SELECT
   $statement = $connexion->query('SELECT * FROM CLIENT');
   # Récupérer le résultat ligne par ligne
   while($client = $statement->fetch(PDO::FETCH_OBJ))
   {
        echo "id : ". $client->id;
        echo "  nom : ". $client->nom;
        echo "  date naissance  : ". $client->date_naissance;
        echo "<br>";
   }

?>
```

Chaque ligne correspondant à un OBJET dont les attributs sont les noms des colonnes de la requête Sql

Modes de récupération des données

- Exemple : Afficher tous les clients avec le mode PDO::FETCH_CLASS

```php
<?php

Class Client
{
    private $id;
    private $nom;
    private $date_naissance;
    private $cin;
    private $date_ajout;

    public function getNom(){return $this->nom;}
    //getters et Setters
}
?>
```

Modes de récupération des données

- Exemple : Afficher tous les clients avec le mode PDO::FETCH_CLASS

```php
<?php
    require_once("connexion.php");
    require_once("Client.php");
    # Exécuter le requête SELECT
    $statement = $connexion->query('SELECT * FROM CLIENT');
    $statement->setFetchMode(PDO::FETCH_CLASS, 'Client');
    # Récupérer le résultat ligne par ligne
    while($client = $statement->fetch())
    {
        echo "  nom : ". $client->getNom();
        echo "<br>";
    }
?>
```

Chaque ligne correspondant à un OBJET de type **Client**

“Requêtes préparées”

Requête préparée

- Requête écrite suivant la syntaxe SQL , elle est de type string
- La requête créée est directement utilisable
- **Exemples :**

```
INSERT INTO Client(nom, cin, date_naissance)
     VALUES('bilel',12121212,'1980-10-23')
```

```
SELECT nom, cin FROM Client WHERE id=1
```

- Les requêtes préparées permettent de créer des requêtes SQL qui ne sont pas directement utilisables mais qui contiennent des paramètres auxquels on peut donner des valeurs différentes en fonction des besoins
- C'est une requête SQL **paramétrée** à l'aide de ***marqueurs nommés*(:nom_marqueur)** ou **interrogatifs (?).**

Requête préparée

- Les requêtes préparées permettent de créer des requêtes SQL qui ne sont pas directement utilisables mais qui contiennent des paramètres auxquels on peut donner des valeurs différentes en fonction des besoins

- C'est une requête SQL **paramétrée** à l'aide de ***marqueurs nommés*(:nom_marqueur)** ou **interrogatifs (?).**

Requête préparée

- Elle est exécutée une ou plusieurs fois en substituant à chaque exécution les marqueurs avec les valeurs d'un tableau :
  - Associatif dont les clés correspondent aux marqueurs nommés.
  - Indicé où les valeurs sont substituées aux marqueurs interrogatifs par ordre d'apparition
- **Exemple:**

```
INSERT INTO Client(nom, cin, date_naissance) VALUES(?,?,?)
```

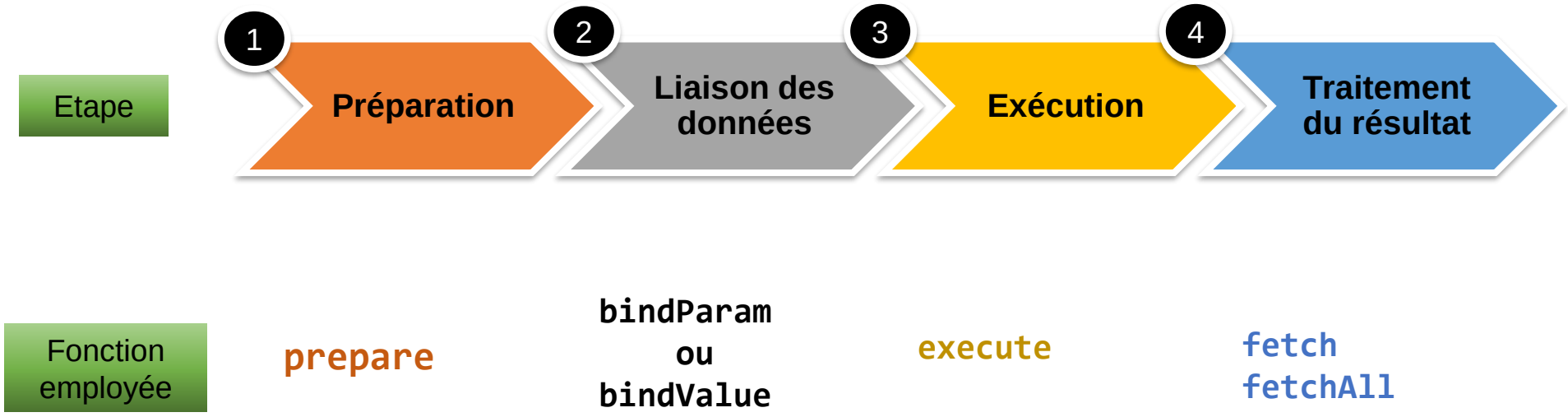
Les marqueurs qui doivent être remplacés par des valeurs avant exécution de la requête

Pourquoi les requêtes préparées

- Lorsqu'il y a de nombreux paramètres dans une requête SQL, on se retrouve avec de nombreuses concaténations de chaines entourées de "cotes" ce qui est une grande source d'erreurs et de lenteurs d'écriture.
  - Les requêtes préparées permettent de bien dissocier la requête des paramètres qui vont lui être fournis avant son exécution.
- Protection automatique des valeurs des paramètres pour interdire les attaques par injection de code SQL
- Amélioration des performances en cas d'exécutions répétées : C'est une requête récurrente, que l'on compile avec des variables, et donc réutilisable (exécutée plusieurs fois) en fournissant les valeurs manquantes.

Les requêtes préparées: Les étapes

- L'exécution d'une requête paramétrée suit 4 étapes:



Etape1: préparation de la requête

- Écrire la chaîne de requête comme paramètre de la méthode **prepare()** de l'objet **PDO**.
- Si le serveur de base de données prépare avec succès la requête, cette méthode retourne un objet de type **PDOStatement** qui représente la requête préparée et qui est noté $reqprep dans notre exemple.
- **Exemples:**

Marqueurs interrogatifs

```
$reqpre = $connexion->prepare
("INSERT INTO Client(nom, cin, date_naissance) VALUES(?,?,?)");
```

Marqueurs nommés

```
$reqpre = $connexion-> prepare("INSERT INTO Client(nom, cin, date_naissance)
          VALUES(:nom,:cin,:dn)");
```

Etape2: liaison des données

- Lier les données (marqueurs) de la requête avec des valeurs ou des variables. Cette liaison peut se faire de plusieurs manières :
  - En utilisant un **tableau** associatif ou d'indices
  - En utilisant la fonction **bindParam**: associe à un marqueur le contenue ***d'une variable***
  - En utilisant la fonction **bindValue**: associe à un marqueur une valeur directement ou le contenue d'une variable

Etape2: liaison des données

En utilisant un tableau associatif ou d'indices

- Il s'agit de créer un tableau associatif si la requête utilise des marqueurs nommées ou un tableau d'indices si la requête utilise des marqueurs interrogatifs. Le tableau a créé sera ensuite passé en paramètre à la méthode `execute($tab)`
- Forme d'un tableau associatif (pour les marqueurs nommés) de la forme : `nom_marqueur=>nom_variable`

```
$tab = array(':nom'=>$nom, ':cin'=>$cin, ':dn' =>$dn);
```

- Forme d'un tableau d'indice (pour les marqueurs ?): **par ordre, la première case sera associée avec le premier marquer ?**

```
$tab = array($nom, $cin,$dn);
```

Etape2: liaison des données

- **Exemple1:** Ajout d'un client avec **marqueurs nommés**

```
# Etape 1 :  récupération des données à partir d'un formulaire
    $nom = $_POST['nom'];      $cin = $_POST['cin'];     $dn = $_POST['dn'];
# Etape 2 :  Préparation de la requête
  $reqpre = $connexion->prepare
("INSERT INTO Client(nom, cin, date_naissance) VALUES(:nom,:cin,:dn)");
#Etape 3 :Liaison entre les marqueurs et les données du formulaire
#Créer un Tableau associatif-> clé :nom du marqeur et valeur : donnée
  $tab = array(':nom'=>$nom, ':cin'=>$cin, ':dn' =>$dn);
# Etape 4 : exécution de la requête
  $resultat = $reqpre->execute($tab);
        if($resultat)
           echo "Ajout avec succès";
        else
            echo "echec d'ajout";
```

Etape2: liaison des données

- **Exemple1:** Ajout d'un client avec **marqueurs interrogatifs**

```
# Etape 1 :  récupération des données à partir d'un formulaire
    $nom = $_POST['nom'];      $cin = $_POST['cin'];     $dn = $_POST['dn'];
# Etape 2 :  Préparation de la requête
  $reqpre = $connexion-> prepare
("INSERT INTO Client(nom, cin, date_naissance) VALUES(?,?,?)");
# Etape 3 :Liaison entre les marqueurs et les données du formulaire
# Créer un Tableau indicé : par ordre, la première case remplacera le premier marqueur,
etc.
  $tab = array($nom, $cin,$dn);
        $resultat = $reqpre->execute($tab);
        if($resultat)
           echo "Ajout avec succès";
        else
            echo "echec d'ajout";
```

Etape2: liaison des données

En utilisant la fonction bindParam

- **`bindParam`** : Lie un paramètre à un nom de variable spécifique
- **Syntaxe :** `bindParam(marqueur,varibale, type);`
- **Exemple 1:**

```
$nom = $_POST['nom'];
$cin = $_POST['cin’];
$dn = $_POST['dn'];
$reqpre = $connexion-> prepare
        ("INSERT INTO Client(nom, cin, date_naissance)
            VALUES(:nom,:cin,:dn)");
//associer un marqueur à une variable
  $reqpre->bindParam(':nom',$nom, PDO::PARAM_STR);
  $reqpre->bindParam(':cin',$cin, PDO::PARAM_INT);
  $reqpre->bindParam(':dn',$dn);
  $resultat = $reqpre->execute();
```

Etape2: liaison des données

En utilisant la fonction bindParam

- **Exemple 2:**

```
$nom = $_POST['nom'];
$cin = $_POST['cin’];
$dn = $_POST['dn'];
$reqpre = $connexion-> prepare
    ("INSERT INTO Client(nom, cin, date_naissance)
            VALUES(?,?,?)");
//associer un marqueur par sa position à une variable
  $reqpre->bindParam(1,$nom, PDO::PARAM_STR);
  $reqpre->bindParam(2,$cin, PDO::PARAM_INT);
  $reqpre->bindParam(3,$dn);

  $resultat = $reqpre->execute();
```

Etape2: liaison des données

- En utilisant la fonction **bindValue**: associe à un marqueur une valeur directement ou le contenue d'une variable

```
bool PDOStatement::bindValue(parameter, value [,int data_type])
```

  - parameter : le paramètre (nom ou position [1…n])
  - value : sa valeur
  - data_type : le type de la valeur
    - PDO::PARAM_BOOL booléen.
    - PDO::PARAM_NULL NULL SQL.
    - PDO::PARAM_INT INTEGER SQL.
    - PDO::PARAM_STR CHAR, VARCHAR ou autre chaîne.
    - PDO::PARAM_LOB "objet large" SQL.

Etape2: liaison des données

- En utilisant la fonction **bindValue**: associe à un marqueur une valeur directement ou le contenue d'une variable
- **Exemple:**

```
$cnx=new PDO ($sdn, $user, $pass);
$reqpre = $cnx->prepare(
                "SELECT * FROM user WHERE User= ?") ;
$reqpre->bindValue(1, 'root') ;
$res= $reqpre->execute() ;
```

Etapes3 &4: exécution &traitement du résultat

Execution

- La fonction `execute` exécute une requête préparée :

```
$reqpre->execute()
```

- Retourne `TRUE` en cas de succès ou `FALSE` si une erreur survient.

Traitement du résultat

- La fonction `$reqpre->fetchAll()` ou `fetch()` permettent de traiter le résultat

Transactions

Transactions

- Une transaction est un ensemble d'actions qui prend la base de données dans un état cohérent et elle la rend dans un autre état cohérent.
- Le principe des transactions est d'effectuer un ensemble d'opérations de manière groupée (plusieurs insertions successives dans différentes tables par exemple)
- Mode PDO par défaut :
  - Chaque requête est validée automatiquement
- **Les fonctions de PDO pour la gestion des transactions**

Fonction	Rôle
beginTransaction()	Une transaction doit commencer par l'appel de cette méthode
commit()	Appelée pour valider l'ensemble des requêtes exécutés au cours de la transaction
rollBack()	Annuler les requêtes exécutées au cours d'une transaction

Transactions

- **Exemple:** ajouter deux comptes pour le même client ou annuler la transaction

```php
<?php
include("connexion.php");
#Démarrer la transaction
$connexion->beginTransaction();
$idclient = 25;
$requete="INSERT INTO COMPTE (solde, type, id_client) values (?,?,?)";
#Préparons la requêt
$requetePrepare = $connexion->prepare($requete);
#Initialiation des données
$solde = 12;
$type="epargne";
#Liaison entre les données et les paramètres de la requête
$requetePrepare1->bindParam(1,$solde);
$requetePrepare1->bindParam(2,$type);
$requetePrepare1->bindParam(3,$idclient);
#Exécuter la requête 1 :
$resultat1 = $requetePrepare->execute();
```

Transactions

```
#Initialiser les données de la deuxième requête
$solde = 300;
$type="courant";
#liaison entre les données et les paramètres de la requête
$requetePrepare->bindParam(1,$solde);
$requetePrepare->bindParam(2,$type);
$requetePrepare->bindParam(3,$idclient);
#Exécuter la requête 2 :
$resultat2 = $requetePrepare->execute();

if($resultat1 && $resultat2) {
    $connexion->commit();
       echo "ajout avec succès";    }
else{
    $connexion->rollBack();
    echo "Echec d'ajout";    }
    ?>
```

Questions?